Part II : reasoning about loops

LN Section 6.3 - 6.7, and 9.3

How to prove this ?

{* P *} **while** g **do** S {* Q *}

- Unfortunately no nice equation for **wp**
- Essentially because we do not know how many times the loop will iterate

$$\textbf{wp}\ (\textbf{while}\ g\ \textbf{do}\ S)\ Q \sim S^{-n}\ Q$$

- But what is n?

Idea

Need a predicate ***I***, an ***invariant***, that holds at the end of every iteration.

$iter_1$: // g // ; **S** {* **I** *}
$iter_2$: // g // ; **S** {* **I** *}
…
$iter_n$: // g // ; **S** {* **I** *} // last iteration
exit : // ¬g //

- Observation: With such an invariant, I /\ ¬g must hold when the loop terminates (after the last iteration).
- The post-condition Q can be established if I /\ ¬g ⟹ Q.

Ok, what kind of predicate is I ??

- We need an I that holds at the end of every iteration. Inductively, this means that we can assume it to hold at the start of the iteration (established by the previous iteration).

- So, it is sufficient to have an I satisfying this property:

$$\{* \; I \wedge g \; *\} \quad S \quad \{* \; I \; *\}$$

Establishing the base case

The previous inductive argument does not however apply for the first iteration, because it has no previous iteration. Let's just insist that this first iteration should also maintain the property:

$$\{* \; I \wedge g \; *\} \quad S \quad \{* \; I \; *\}.$$

Something else is now needed to guarantee that the first iteration can indeed assume I as its pre-condition. We can establish this by requiring that the original precondition P implies this I.

To Summarize

- Capture this in an inference rule (Rule 6.3.2):

$$\frac{P \Rightarrow I \qquad \{* \; g \wedge I \; *\} \;\; S \;\; \{* \; I \; *\} \qquad I \wedge \neg g \Rightarrow Q}{\{* \; P \; *\} \;\; \textbf{while} \;\; g \;\; \textbf{do} \;\; S \;\; \{* \; Q \; *\}}$$

// setting up I
// invariance
// exit cond

Taking P to be I, we get:

$$\frac{\{* \; g \wedge I \; *\} \;\; S \;\; \{* \; I \; *\} \qquad I \wedge \neg g \Rightarrow Q}{\{* \; I \; *\} \;\; \textbf{while} \;\; g \;\; \textbf{do} \;\; S \;\; \{* \; Q \; *\}}$$

Few things to note

- This rule is only good for **partial correctness**

- If you have a program with the statement

while g **do** S

and you find an I such that {* I *} **while** g **do** S {* Q *},

then you can compose this with the calculations of wp for the other statements in the program. Then you will get something **sound** but not **complete.**

$$\frac{\{*\ g \wedge I\ *\}\quad S\quad \{*\ I\ *\} \qquad I \wedge \neg g \Rightarrow Q}{\{*\ I\ *\}\ \textbf{while}\ g\ \textbf{do}\ S\ \{*\ Q\ *\}}$$

Examples

- Prove the (partial) correctness of the following loop:

{* true *} **while** i≠n **do** i++ {* i=n *}

- Need an I such that
  - true ⟹ I
  - {* i≠n ∧ I *} i++ {* I *}
  - I ∧ i=n ⟹ i=n
- I := true works!

$$\frac{P \Rightarrow I \quad \{* g \wedge I *}\ S\ \{* I *\} \quad I \wedge \neg g \Rightarrow Q}{\{* P *\}\ \textbf{while}\ g\ \textbf{do}\ S\ \{* Q *\}}$$

Examples

- Prove the (partial) correctness of the following loop:

{* i=0 /\ n=10 *} **while** i<n **do** i++ {* i=n *}

- Need an I such that
  - i=0 /\ n=10 $\Longrightarrow$ I
  - {* i<n $\wedge$ I *} i++ {* I *}
  - I $\wedge$ i $\geq$ n $\Longrightarrow$ i=n
- I := i $\leq$ n works!

$$\frac{P \Rightarrow I \quad \{* g \wedge I *}\ S\ \{* I *\} \quad I \wedge \neg g \Rightarrow Q}{\{* P *\}\ \textbf{while}\ g\ \textbf{do}\ S\ \{* Q *\}}$$

Examples

- Prove the (partial) correctness of the following loop:

{* i=0 /\ s=0 /\ n=10 *} **while** i<n **do** { s = s+2 ; i++ } {* s=20 *}

- Need an I such that
  - i=0 /\ s=0 /\ n=10 $\Longrightarrow$ I
  - {* i<n $\wedge$ I *} s = s+2 ; i++ {* I *}
  - I $\wedge$ i $\geq$ n $\Longrightarrow$ s=20
- I := i $\leq$ n $\wedge$ s = 2i $\wedge$ n = 10 works!

$$\frac{P \Rightarrow I \quad \{* g \wedge I *}\ S\ \{* I *\} \quad I \wedge \neg g \Rightarrow Q}{\{* P *\}\ \textbf{while}\ g\ \textbf{do}\ S\ \{* Q *\}}$$

Proving termination

- Again, consider:

{* P *} while g do S {* Q *}

- Idea: get an integer expression m, called "**termination metric**", satisfying :
  1. At the start of every iteration m > 0
  2. Each iteration decreases m

- Since m has a lower bound (condition 1), it follows that the loop cannot iterate forever. In other words, it will terminate.

Capturing the termination conditions

- At the start of every iteration m > 0 :
  - $I \wedge g \Rightarrow m > 0$
- Each iteration decreases m :

  {* I /\ g *} C:=m; S {* m<C *}

To Summarize

- Rule B.1.8:

$$
\begin{array}{l}
P \Rightarrow I \qquad \text{// setting up I} \\
\{* \; g \wedge I \; *\} \quad S \quad \{* \; I \; *\} \qquad \text{// invariance} \\
I \wedge \neg g \Rightarrow Q \qquad \text{// exit cond} \\
\{* \; I \wedge g \; *\} \quad C := m; \; S \quad \{* \; m < C \; *\} \qquad \text{// m decreasing} \\
I \wedge g \Rightarrow m > 0 \qquad \text{// m bounded below} \\
\hline
\{* \; P \; *\} \quad \textbf{while} \; g \; \textbf{do} \quad S \quad \{* \; Q \; *\}
\end{array}
$$

Alternatively it can be formulated as follows

- Rule B.1.7

$$\frac{\{*\ g\ /\backslash\ I\ *\}\quad S\quad \{*\ I\ *\} \qquad I\ /\backslash\ \neg g\ \Rightarrow\ Q \qquad \{*\ I\ /\backslash\ g\ *\}\quad C:=m;\ S\quad \{*\ m<C\ *\} \qquad I\ /\backslash\ g\ \Rightarrow\ m > 0}{\{*\ I\ *\}\ \underline{\text{while}}\ g\ \underline{\text{do}}\ S\ \{*\ Q\ *\}}$$

// invariance
// exit cond
// m decreasing
// m bounded below

- This is a special instance of B.1.8. On the other hand, together with the pre-condition strengthening rule it also implies B.1.7.

Examples

- Prove that the following program terminates:

{* i≤n *} **while** i≠n **do** i++ {* true *}

- Need an I and m such that
  - i≤n ⟹ I
  - {* i≠n ∧ I *} i++ {* I *}
  - I ∧ i=n ⟹ true
  - {* I ∧ i≠n *} C:=m ; i++ {* m < C *}
  - I ∧ i≠n ⟹ m > 0
- I = i≤n works!
- m = n – i works!

$P \Rightarrow I$
$\{* g \wedge I *\}$ S $\{* I *\}$
$I \wedge \neg g \Rightarrow Q$
$\{* I \wedge g *\}$ C:=m; S $\{* m<C *\}$
$I \wedge g \Rightarrow m > 0$

$\{* P *\}$ **while** g **do** S $\{* Q *\}$

Examples

- Prove that the following program terminates:

```
{*  candy=100 /\ apple=100 }
 while candy>0 \/ apple>0  do
    {
     if candy>0 then { candy--  ; apple += 2 }
     else  { apple-- }
}
{* true *}
```

$$\frac{P \Rightarrow I \quad \{* g \wedge I *\}\ S\ \{* I *\} \quad I \wedge \neg g \Rightarrow Q \quad \{* I \wedge g *\}\ C:=m;\ S\ \{* m<C *\} \quad I \wedge g \Rightarrow m > 0}{\{* P *\}\ \textbf{while}\ g\ \textbf{do}\ S\ \{* Q *\}}$$

- Need an I and m such that
  - candy=100 /\ apple=100 $\Longrightarrow$ I
  - {* candy>0 \/ apple>0 $\wedge$ I *} S {* I *}
  - I $\wedge$ candy $\leq$ 0 $\wedge$ apple $\leq$ 0 $\Longrightarrow$ true
  - {* I $\wedge$ candy>0 \/ apple>0 *} C:=m ; S {* m < C *}
  - I $\wedge$ candy>0 \/ apple>0 $\Longrightarrow$ m > 0

- I = candy + apple $\geq$ 0
- m = 3 candy + apple

A bigger example

- Let's now consider a less trivial example. The following program claims to check if the array segment a[0..n) consists of only 0's. Prove it :

```
{* 0≤n *}
   i := 0 ; r := true ;
   while  i<n  do  {  r := r /\ (a[i]=0)   ;   i++ }
{*  r  = (∀k : 0≤k<n : a[k]=0) *}
```

- You will need to propose an invariant and a termination metric.
- This time, the proof will be quite involved. The rule to handle loops also generates multiple conditions.

Invariant and termination metric

```
{* 0≤n *}
   i := 0 ; r := true ;
  while  i<n  do  {  r := r /\ (a[i]=0)   ;   i++ }
{*  r  = (∀k : 0≤k<n : a[k]=0) *}
```

- Let's go with this choice of invariant I :

I_1 $(r = (\forall k : 0 \le k < i : a[k]=0))$ $\wedge$ $0 \le i \le n$ I_2

- m = n - i

$P \Rightarrow I$
$\{* g \wedge I *\}$ S $\{* I *\}$
$I \wedge \neg g \Rightarrow Q$
$\{* I \wedge g *\}$ C:=m; S $\{* m<C *\}$
$I \wedge g \Rightarrow m > 0$

$\{* P *\}$ **while** g **do** S $\{* Q *\}$

Invariant and termination metric

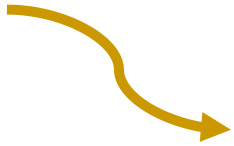
```
{* 0≤n *}
   i := 0 ; r := true ;
  while  i<n  do  {  r := r /\ (a[i]=0)   ;   i++ }
{*  r  = (∀k : 0≤k<n : a[k]=0) *}
```

- Let's go with this choice of invariant I :

I_1 : $(r = (\forall k : 0 \le k < i : a[k]=0))$ $\wedge$ I_2 : $0 \le i \le n$

$P \Rightarrow I$
$\{* g \wedge I *\}$ S $\{* I *\}$
$I \wedge \neg g \Rightarrow Q$
$\{* I \wedge g *\}$ C:=m; S $\{* m<C *\}$
$I \wedge g \Rightarrow m > 0$

$\{* P *\}$ **while** g **do** S $\{* Q *\}$



$\{* P *\}$ initialization $\{* I *\}$
$\{* g \wedge I *\}$ S $\{* I *\}$
$I \wedge \neg g \Rightarrow Q$
$\{* I \wedge g *\}$ C:=m; S $\{* m<C *\}$
$I \wedge g \Rightarrow m > 0$

$\{* P *\}$ **while** g **do** S $\{* Q *\}$

Invariant and termination metric

So we actually want to prove:

```
{* 0≤n *}
   i := 0 ; r := true ;
{* (r  = (∀k : 0≤k<i : a[k]=0))   /\   0≤i≤n *}
  while  i<n  do  {  r := r /\ (a[i]=0)   ;   i++ }
{*  r  = (∀k : 0≤k<n : a[k]=0) *}
```

We need to prove

1. (Exit Condition) $I \wedge \neg g \Rightarrow Q$

2. (Initialization Condition)
   {* given-pre-cond *} i := 0 ; r := **true** {* I *},

3. (Invariance) {* I /\ g *} body {* I *}

   Or equivalently, prove: I /\ g $\Rightarrow$ **wp** body I

4. (Termination Condition) {* I /\ g *} (C:=m ; body) {* m<C *}
5. (Termination Condition) I /\ g $\Rightarrow$ m>0

Reformulating them in terms of wp

- Several statements are Hoare triples.
- If they have no loops, we can reformulate in terms of wp.

1. (Exit Condition) $I \wedge \neg g \Rightarrow Q$
2. (Initialization Condition)
   given-pre-cond $\Rightarrow$ **wp** (i := 0 ; r := **true**) I
3. (Invariance) $I \wedge g \Rightarrow$ **wp** body I
4. (Termination Condition) $I \wedge g \Rightarrow$ **wp** (C:=m ; body) (m<C)
5. (Termination Condition) $I \wedge g \Rightarrow m>0$

Now we can use the previous proof system for predicate logic to prove those implications.

Top level structure of the proofs

- **PROOF** PEC (proof of the exit condition)
  [A1] r = (∀k : 0≤k<i : a[k]=0) (I_1)
  [A2] 0≤i≤n (I_2)
  [A3] i≥n (¬g)
  [G] r = (∀k : 0≤k<n : a[k]=0) (Q)
  .... (the proof itself)
  END

- **PROOF** PInit (proof of the initialization condition)
  [A1] 0≤n (P)
  [G] (true = (∀k : 0≤k<0 : a[k]=0)) /\ 0≤0≤n (**wp** (i := 0 ; r := **true**) I)
  ...
  END

Proof of the initialization condition

PROOF PInit
[A1] 0≤n
[G] (true = (∀k : 0≤k<0 : a[k]=0)) /\ 0≤0≤n
BEGIN

1. { follows from A1 } 0≤0≤n
2. { see subproof } true = (∀k : 0≤k<0 : a[k]=0)

EQUATIONAL PROOF
(∀k : 0≤k<0 : a[k]=0)
= { the domain is false }
(∀k : false : a[k]=0))
= {∀ over empty domain }
true
END

3. { conjunction of 1 and 2 } G

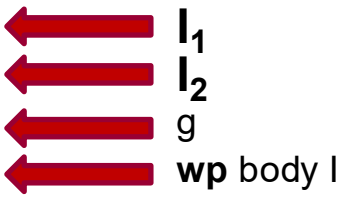
end

Proof of I's invariance

PROOF PIC (proof of the invariance condition)
[A1] r = (∀k : 0≤k<i : a[k]=0)
[A2] 0≤i≤n
[A3] i<n
[G1] r/\(a[i]=0) = (∀k : 0≤k<i+1 : a[k]=0)
[G2] 0≤i+1≤n
BEGIN
1. { see the subproof below } G1

EQUATIONAL PROOF
(∀k : 0≤k<i+1 : a[k]=0)
= { follows from A2 }
(∀k : 0≤k<i \/ k=i : a[k]=0)
= { ∀ domain-split }
(∀k : 0≤k<i: a[k]=0) /\ (∀k : k=i : a[k]=0)
= { A1 }
r /\ (∀k : k=i : a[k]=0)
= { quant. over singleton }
r /\ (a[i]=0)
END

2. { A2 implies 0≤i+1 and A3 implies i+1≤n } G2
END



Top level structure of the proofs of termination

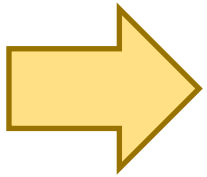
- **PROOF** TC1 (proof of the 1st termination condition)
  [A1] r = ($\forall$k : 0≤k<i : a[k]=0) (I_1)
  [A2] 0≤i≤n (I_2)
  [A3] i<n (g)
  [G] n - (i+1) < n-1 (**wp** (C:=m ; body) (m<C))
  ....
  END

- **PROOF** TC2
  [A1] r = ($\forall$k : 0≤k<i : a[k]=0) (I_1)
  [A2] 0≤i≤n (I_2)
  [A3] i<n (g)
  [G] n - i > 0 (I /\ g $\Rightarrow$ m>0)
  ....
  END

Loop with breaks, LN Sec. 9.3

- There is no “break” statement in uPL, but we can still look at the concept behind it, namely to stop a loop earlier because we know that it has achieved its objective.

```
i := 0 ; r := true ;
while  i<n do  {
    r := r /\ (a[i]=0)  ;
    i++
}
```



```
i := 0 ; r := true ;
while  i<n /\ ¬r  do  {
    r := r /\ (a[i]=0)  ;
    i++
}
```

- How to prove that breaking the loop early is indeed safe?

Loop with breaks

- Consider the following loop, which has been proven correct using an **invariant I** and a **termination metric m**.

(1) {* P *} **while** g **do** S {* Q *}

- Suppose now we optimize the loop by adding a ”break” to let it terminate early in some situation:

(2) {* P *} **while** g **/\ h do** S {* Q *}

- The validity of (1) implies:
  - (2) will also terminate
  - (2) will also maintain I’s invariance
  - If g becomes false, (2) will terminate in Q

Loop with breaks

(2) {* P *} **while** g /\ **h** **do** S {* Q *}

So to prove that (2) is valid, given (1) from the previous slide, we only need to prove that when (2) terminates early because h becomes false, then it will also terminate in Q. In other words, we only need to prove:

$$I \wedge g \wedge \neg h \Rightarrow Q$$

where I is the invariant used in (1).

Example

```
{* 0≤n *}

i := 0 ; r := true :
while  i<n  /\ r  do  {
    r := r /\ (a[i]=0)  ;
    i++
}

{*  r  = (∀k : 0≤k<n : a[k]=0) *}
```

We have proven the basic form of this program (without the break with /\ r) before. Now prove that with the break the program is still correct.

Dealing with more constructs

LN Section 6.10 -- 6.12

Simultant assignment

$$x,y := e_1,e_2$$

- Simultant assignment $x,y := e_1,e_2$ evaluates e_1 and e_2 in the current state to values v_1 and v_2, then assign these to x and y respectively.

$$\mathbf{wp}\ (x,y := e_1,e_2)\ Q \ = \ Q[e_1,e_2/x,y]$$

- Simultant substitution $Q[e_1,e_2/x,y]$ replaces any free occurrence of x in Q with e_1, and y with e_2. In particular, we should **not** do the substitution of y **after** the substitution of x, nor the other way around.

The order of assignment matters

- The order in which you assign variables matters (you already know this). E.g. these three have different behavior:

wp (x:=x*y; y:=x+y) (x=2 /\ y=1) = xy=2 /\ xy+y=1

equivalent to: x=-2 /\ y=-1

wp (y:=x+y; x:=x*y) (x=2 /\ y=1) = x(x+y)=2 /\ x+y=1

equivalent to: x=2 /\y=-1

wp (x,y := x*y, x+y) (x=2 /\ y=1) = xy=2 /\ x+y=1

which has no integer solution.

Assignment into an array

- Recall :

wp (x:=e) Q = Q[e/x]

- Unfortunately this rule does not extend to $a[e_1] := e_2$. Consider these examples, where the pre-condition is obtained by literally applying the above calculation rule:

(valid pre-cond) {* y = 10 *} a[i] := y {* a[i] = 10 *}

(not a valid pre-cond) {* a[0] = 10 *} a[i] := y {* a[0] = 10 *}

Fix : {* (i=0 → y | a[0]) = 10 *}

The formalism

- Let a be an array. Introduce this notation to mean the same array as a, except its i-th element, which is e:

$$a\ (i\ \textbf{repby}\ e)$$

- Formally defined via this equation:

$$a(i\ \textbf{repby}\ e)[k] \quad = \quad i=k \rightarrow e \mid a[k]$$

- Treat array assignments like a[i] := e as the assignment

$$a\ :=\ a(i\ \textbf{repby}\ e)$$

This has the same effect, but now we can safely use the old wp rule for assignment.

Example

- Prove: {* i≠k *} a[k]:= 0 ; a[i]:=1 {* a[k] = 0 *}
- Let's first calculate the **wp**:

{* i=k → 1 | 0 = 0 *} // def. repby

{* (1=k → 1 | a(k repby 0)[k]) = 0 *} // wp

a[k]:=0 // a := a(k repby 0)

{* (i=k → 1 | a[k]) = 0 *} // def. repby

{* a(i repby 1)[k] = 0 *} // wp

a[i]:=1 // a := a(i repby 1)

{* a[k] = 0 *}

- Now we need to prove that i≠k implies the wp.

Reducing a program spec to a statement spec

- Consider the program below with the given specification.

{* x > 0 *} P(x:int) { x++ ; **return** x } {* return = x+1 *}

Note: “return” in the post-condition is a special variable that refers to the P’s return value.

- Since we define our Hoare logic at the statement level, we first need to ”reduce” such a program-level specification to a specification in terms of its body.
- Such reduction raises several issues:
  - What is the logic of a ”return” statement ?
  - Should x in the poscond refer to its initial or final value?
  - How to refer to the “old” value of a variable?

Some restrictions imposed on uPL

- First, it is useful to be aware of some restrictions imposed on uPL which were introduced to simplify the resulting Hoare logic, allowing us to focus our study on its main concepts.
  - "**return**" statement should only appear as the last statement in the program.
  - Formulas in the pre- and post-conditions of a program-level specification of a program P can only refer to P's parameters or to "auxiliary variables" (explained later).
  - Parameters are either **passed by value**, or **passed by copy-restore**. In particular uPL does not allow references/pointers to be passed to a program.

Pass by copy-restore

- Arrays are by default passed by copy-restore. A parameter with “OUT” marker is passed by copy-restore. Example:

```
P(OUT  x, OUT y) { x := T ; y := F }
```

- The behavior of a call e.g. “P(a,b)” is as follows:

  The value of a and b are evaluated, and copied to P’s local variables x and y. The the body of P is executed. Then the final values of x and y are copied to a and b, in that order.

Pass by copy-restore, why?

- Why don't we just use pass-by-reference? Well, allowing this could create "**aliases**" (different variables pointing to the same data-cell). This complicate complicate the logic of assignment. Recall this logic:

$$\{*\ P\ *\}\ \ x{:=}e\ \ \{*\ Q\ *\} \quad = \quad P \Rightarrow Q[e/x]$$

  This assignment will now also affect the meaning of x's aliases! The above logic is too simplistic to handle this.
- In this course we want to avoid this complication, to focus more on the basic Hoare logic.
- Pass-by-copy-restore would still allow us to simulate programs with side effect, which is an interesting class to consider.

Meaning of variables in the post-condition

- If “y” is a pass-by-copy-restore parameter, in the post-condition it refers to its final value. Example:

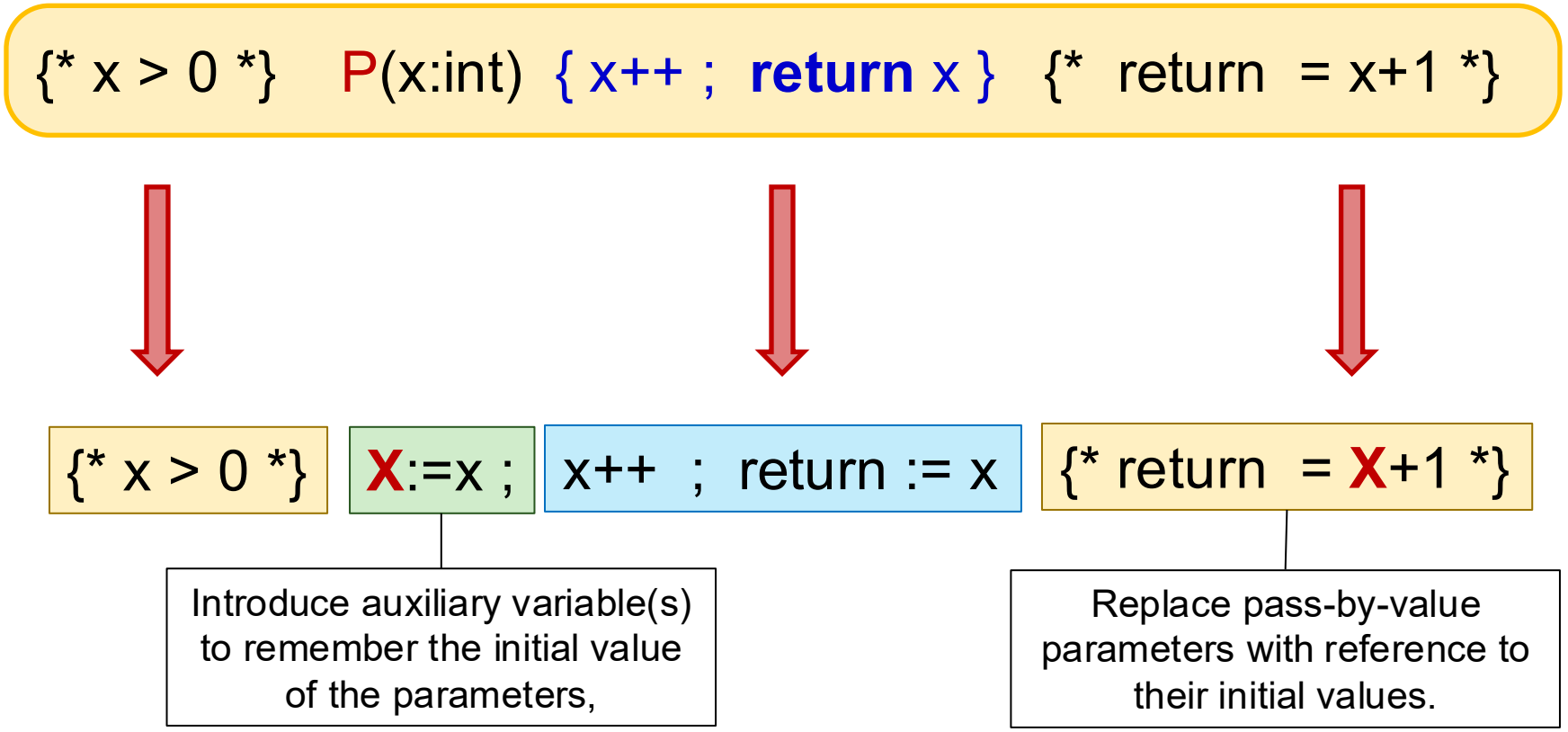
{* true *} **P**(**OUT** y) { y := 1 } {* y>0 *}

- If ”x” is a pass-by-value parameter, in the post-condition it refers to its **initial value.** Example:

{* true *} **P**(x, **OUT** y) { y := x ; x:=0 } {* y=x *}

Example of the reduction

- Consider again the previous example. To reduce the specification to the statement level we will do the following transformation:

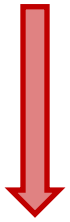


How to refer to parameters' old value?

- Consider a program P(OUT x) that increases the value of x by one. How to write a specification of this P? Again, we will use an auxiliary variable to remember x's old value:

```
{* true *}
    X:=x ; P(OUT x) { x := x+1 }
{* x = X+1 *}
```

Here, **X** is an **auxiliary variable**. It is not part of the program P. It is introduced just for the purpose of expressing P's specification.



```
{* true *}  X:=x ; x := x+1   {* x = X+1 *}
```

The corresponding statement-level specification.

Note: unstructured programs

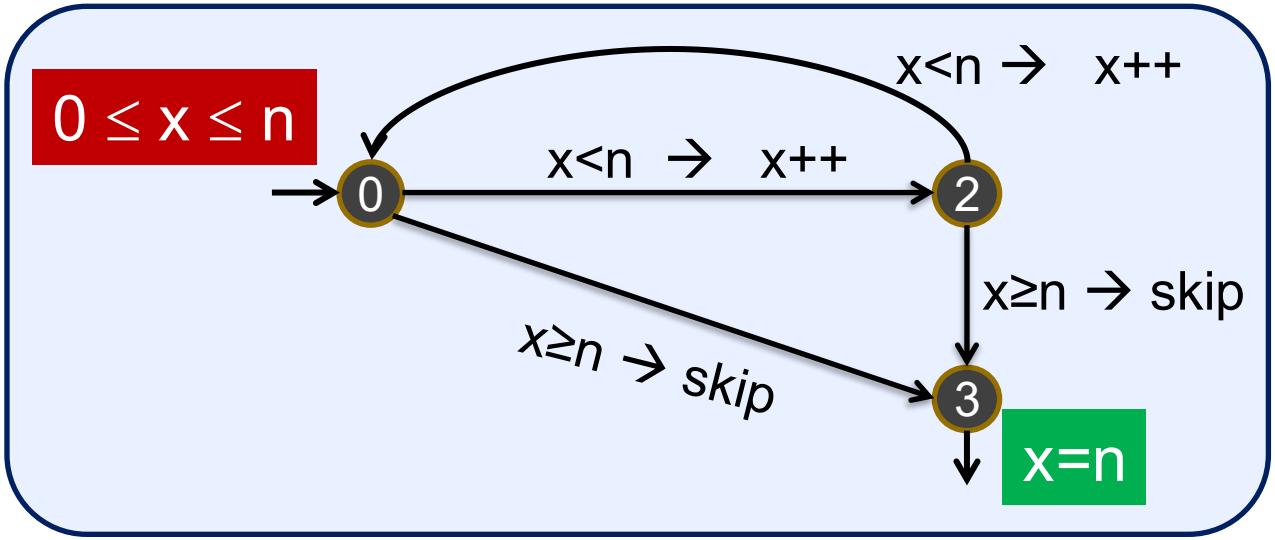
- uPL programs are "**structured**" programs. In such a program, the control flow follows the program's syntax.
- Most programs you write in an imperative language like Java or C# are structured.
- Unstructured programming on the other hand, allows arbitrary jump of control. Example:

```
start: if y=0 then goto exit ;
       if x>y then x:=x-y ; goto start
exit:  ...
```

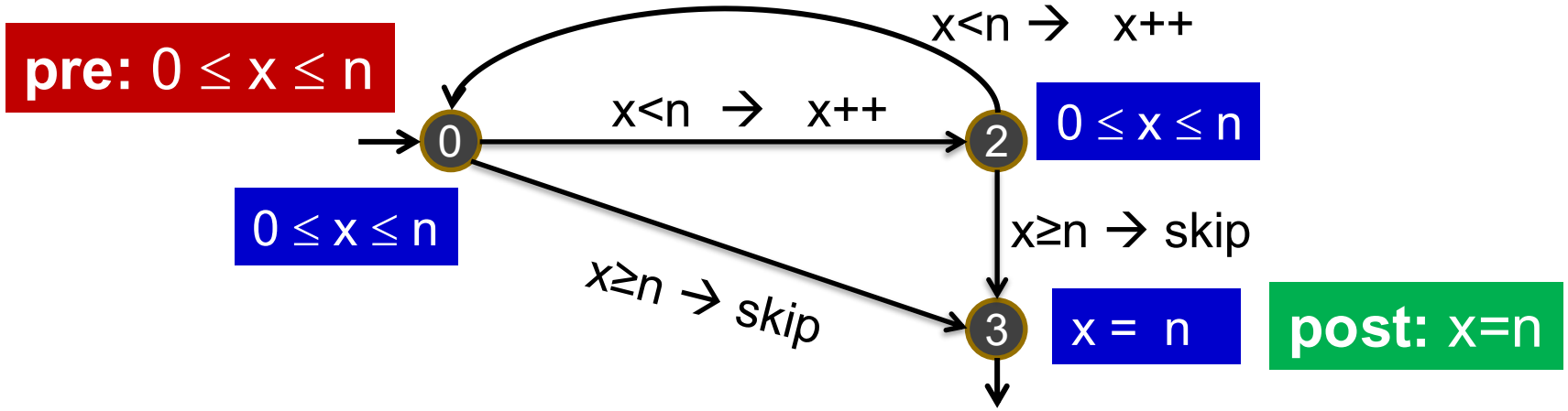
- Hoare logic does **not** work on unstructured programs. E.g. the rule for sequential composition would be broken!

Floyd Logic

Consider the following example of an unstructured program represented by a graph. The program start in node 0, and 3 is the exit node. Each arrow represent an assignment to execute, guarded by the corresponding condition.
The red and green boxes specify the pre- and post-condition of this program.



Floyd Logic



We decorate every node with an assertion (blue). Let A_s be the assertion at node s. Given a pre- and post-condition P and Q, they are valid if:

1. The assertions are consistent. That is: for every arrow $s \xrightarrow{a} t$, $\{* A_s *\}\ a\ \{* A_t *\}$ holds.
2. P implies the assertion at the starting node.
3. The assertion at the exit node implies Q.
4. There is a termination metric m, with a lower bound 0. Executing any arrow decreases this m.